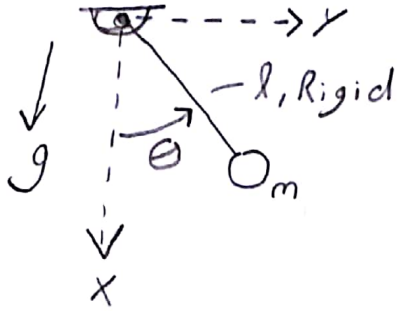
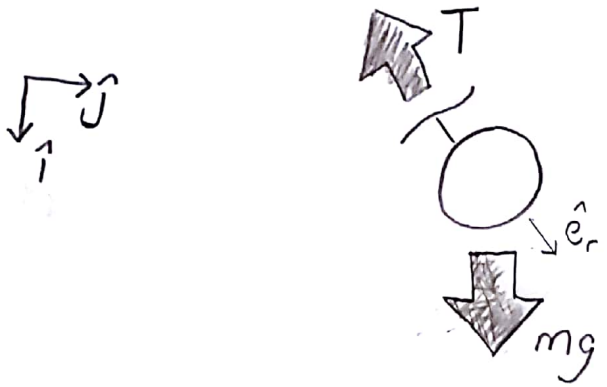
Q)9 Pg.1 In cartesian coordinates develop 3 2nd order ODEs for a simple Pendulum. Similate in Matlab to detect error in satisfying kinematic constraint.

Sketch



1) Develop FBD



2) Apply LMB

$$\Sigma F = m\vec{a}$$

$$-T\hat{e}_r + mg\hat{i} = m\ddot{x}\hat{i} + m\ddot{y}\hat{j}$$

2a) write $\hat{e}_r$ in terms of $\hat{i}$ and $\hat{j}$

$$\hat{e}_r = \frac{x\hat{i} + y\hat{j}}{\sqrt{x^2+y^2}} \quad \bigstar$$

2b) sub' $\bigstar$ back into LMB eqn'

$$\bigstar\bigstar \left[-T\left(\frac{x\hat{i}+y\hat{j}}{\sqrt{x^2+y^2}}\right) + mg\hat{i} = m\ddot{x}\hat{i} + m\ddot{y}\hat{j}\right]$$

Q9)
Pg 2

2c) dot ★★ w/ $\hat{i}$ to get a scalar eqn'

$$\Rightarrow -T\frac{x}{\sqrt{x^2+y^2}} + mg = m\ddot{x} \quad (1)$$

2d) dot ★★ w/ $\hat{j}$ to get 2nd scalar eqn'

$$\Rightarrow -T\frac{y}{\sqrt{x^2+y^2}} = m\ddot{y} \quad (2)$$

3) derive 3rd ODE from a constraint eqn

Constraint: $x^2 + y^2 = l^2$

3a) take 2 derivatives of constraint

$$\frac{d}{dt}(x^2+y^2=l^2) = 2\dot{x}x + 2\dot{y}y = 0 \Rightarrow \dot{x}x + \dot{y}y = 0$$

$$\frac{d}{dt}(\dot{x}x + \dot{y}y) = \ddot{x}x + \dot{x}^2 + \ddot{y}y + \dot{y}^2 = 0$$

$$\Rightarrow x\ddot{x} + y\ddot{y} = -\dot{x}^2 - \dot{y}^2 \quad (3)$$

4) rewrite eqn's (1), (2), (3) to incorporate all terms

★★★
$$\begin{cases} (1) \Rightarrow m\ddot{x} + 0\ddot{y} + \frac{x}{\sqrt{x^2+y^2}}T = mg \\ (2) \Rightarrow 0\ddot{x} + m\ddot{y} + \frac{y}{\sqrt{x^2+y^2}}T = 0 \\ (3) \Rightarrow x\ddot{x} + y\ddot{y} + 0T = -\dot{x}^2 - \dot{y}^2 \end{cases}$$

Q9)

Pg.3 5) ★★★ can be writen in Matrix form

$$\underbrace{\begin{bmatrix} m & 0 & \frac{x}{\sqrt{x^2+y^2}} \\ 0 & m & \frac{y}{\sqrt{x^2+y^2}} \\ x & y & 0 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ T \end{bmatrix}}_{\vec{\omega}} = \underbrace{\begin{bmatrix} mg \\ 0 \\ -\dot{x}^2 - \dot{y}^2 \end{bmatrix}}_{B}$$

6) Solve for $\vec{\omega}$ in Matlab using ODE 45

$$\vec{\omega} = A^{-1} B$$

Note: above eqn can be solved using "\" command

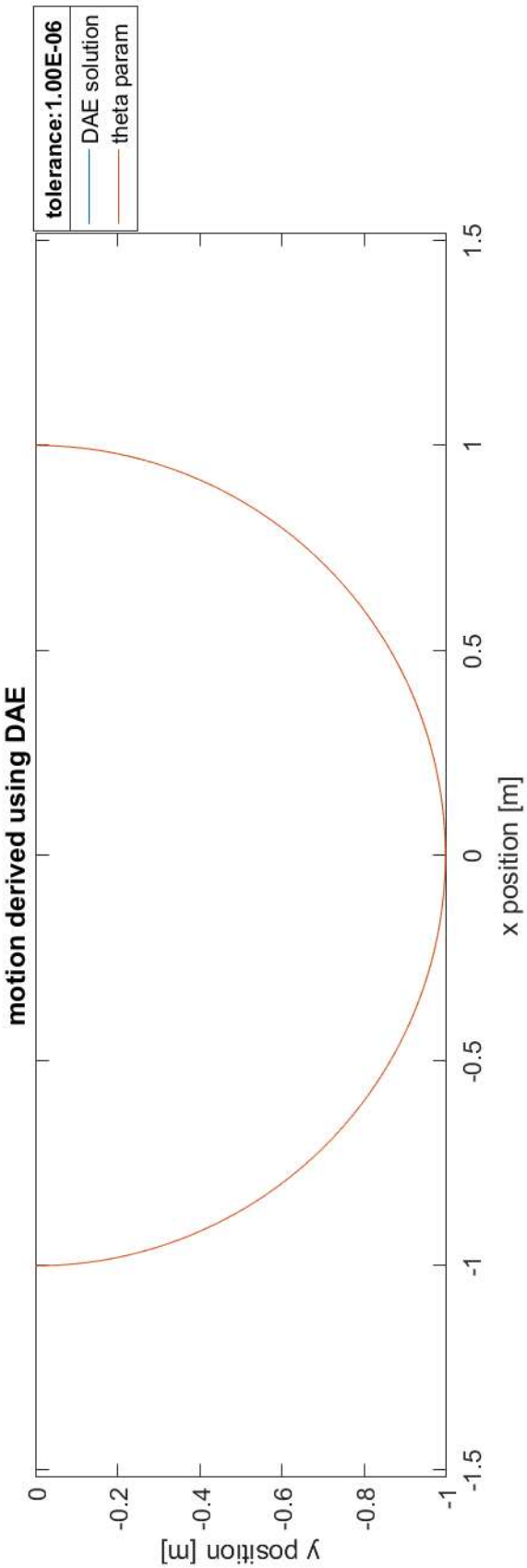
$$\vec{\omega} = A \backslash B$$

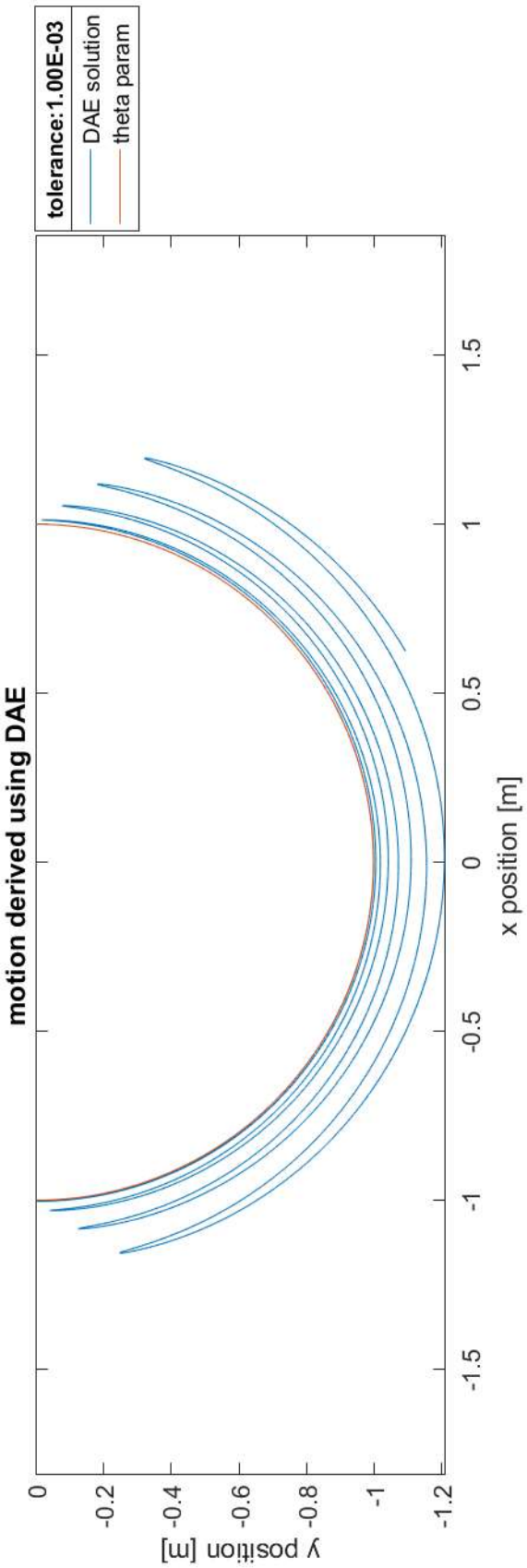
```
%{
Jose Nino
Q19) derive 3 2nd order DAE's for a simple pendulum in
cartesian coordinates.

%}

%set constants
p.m = 1; %mass of pendulum[kg]
p.l = 1; %length of pendulum[m]
p.g = 10; %gravitational accleration near earth[m/s^2]

%set initial conditions
theta0 = pi/2; %initial angle in rads
x0 = p.l*cos(theta0); %[m] initial x position
y0 = sqrt(p.l^2-x0^2); %[m] initial y position
vx0 = 0; %[m/s] initial velocity in the x component
vy0 = 0; %[m/s] initial velocity in the y component

%gather states
q = [x0 vx0 y0 vy0];

% set duration and number of samples
tf = 10; %fin time [sec]
n = 1000; %number of data points wanted

%high tolerances
opts.RelTol = 1e-6;
opts.AbsTol = 1e-6;
%low tolerances
opts2.RelTol = 1e-3;
opts2.AbsTol = 1e-3;

%call ode45 and RHS3 for Pendulum DAEs for high tolerance
[time, states] = ode45(@RHS3,linspace(0,tf,n),q,opts,p);
%call ode45 and RHS3 for Pendulum DAEs for low tolerance
[time2, lowStates] = ode45(@RHS3,linspace(0,tf,n),q,opts2,p);

%call ode45 and RHS4 for minimal coordinate solver for comparison
[time3, states2] = ode45(@RHS4,linspace(0,tf,n),[theta0, 0],opts,p);

%unpack states
%states from DAE
%states with high tolerances
x = states(:,1);
y = states(:,3);
%states with low tolerances
xLow = lowStates(:,1);
yLow = lowStates(:,3);
```

```
%states from theta param
theta = states2(:,1);
x_simple = p.l*cos(theta);
y_simple = p.l*sin(theta);

%plot solution of the pendulum derived using DAE and compare with standard
%circle
%first plot solution with high tolerance and half circle to see drift of
%constraints
subplot(2,1,1);
plot(y,-x);
title('motion derived using DAE');
xlabel('x position [m]');
ylabel('y position [m]');
hold on
plot(y_simple,-x_simple);
hold off;
axis('equal');
lgd = legend('DAE solution','theta param');
title(lgd,strcat('tolerance', ': ' ,num2str(opts.RelTol,'%.2E')));


%second plot graph with low tolerance nd half circle to see drift of
%constraints
subplot(2,1,2);
plot(yLow,-xLow);
title('motion derived using DAE');
xlabel('x position [m]');
ylabel('y position [m]');
hold on
plot(y_simple,-x_simple);
hold off;
axis('equal');
lgd2 = legend('DAE solution','theta param');
title(lgd2,strcat('tolerance', ': ' ,num2str(opts2.RelTol,'%.2E')));


% %compare constraint error based on tolerances
fig4 = figure;
plot(time, abs(p.l^2-y.^2-x.^2),'linewidth',2);
title('Error in constraint');
xlabel('time [sec]');
ylabel('l^2-x^2-y^2  [m]');
hold on
plot(time, abs(p.l^2-yLow.^2-xLow.^2),'linewidth',2);
title('Error in constraint');
xlabel('time [sec]');
ylabel('l^2-x^2-y^2  [m]');
lgd3 = legend(strcat('tolerance', ': ' ,num2str(opts.RelTol,'%.2E')),strcat↙
('tolerance', ': ' ,num2str(opts2.RelTol,'%.2E')));
```

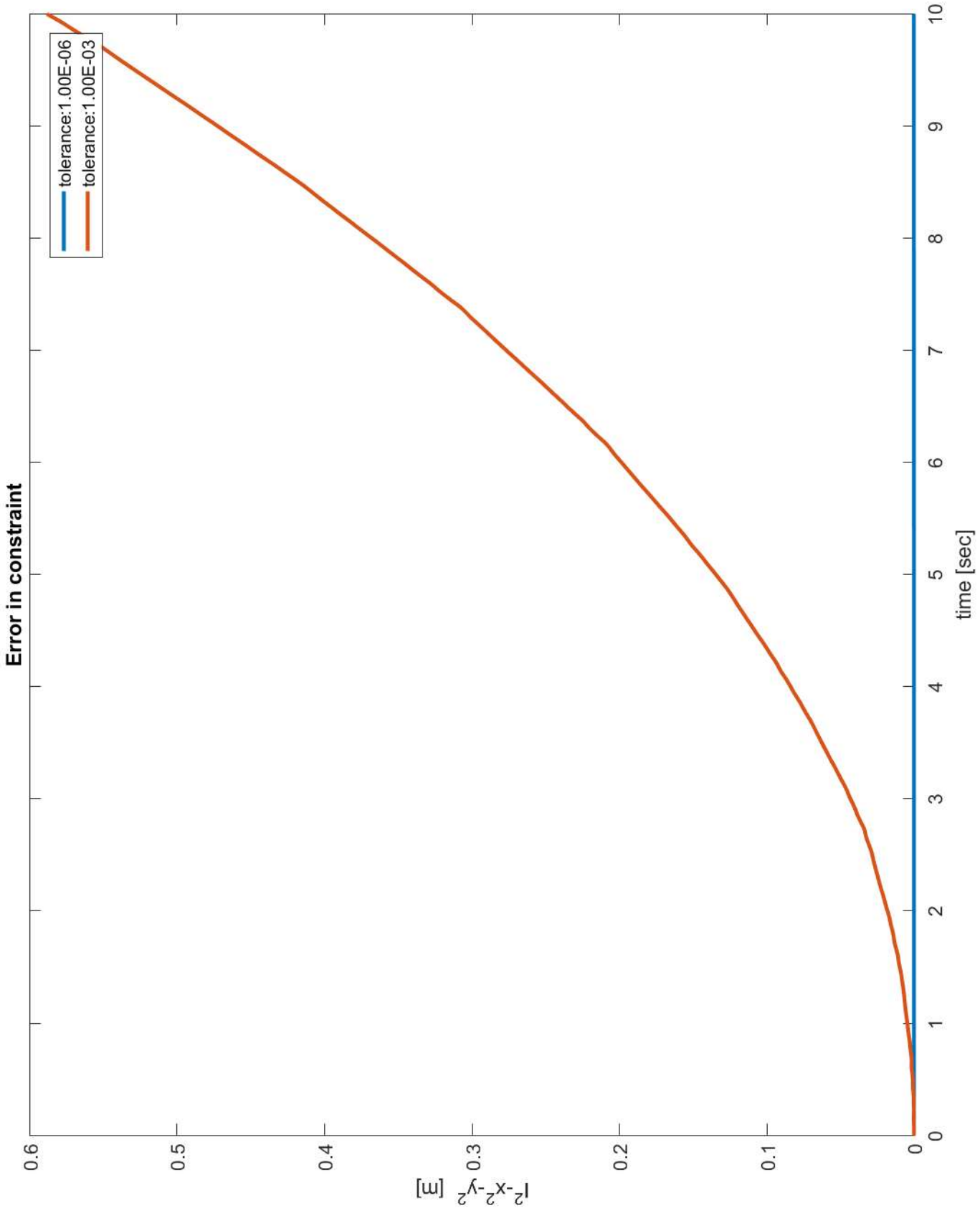
```
%{
Jose Nino
Q19 RHS file for DAE approach
%}

function qdot = RHS3(t,q,p)
%rename states to better understand equations
x = q(1);
vx = q(2);
y = q(3);
vy = q(4);

%set up matrix for the DAE equations found during analysis
A = [p.m, 0, x/(x^2+y^2);0, p.m, y/(x^2+y^2);x, y, 0];
B = [p.m*p.g;0;-vx^2-vy^2];

%solve for x accleration, y accleration, and tension with "\" command
z = A\B;

%build qdot matrix to send to ode45 for integration, ignore tension
qdot(1) = vx;
qdot(2) = z(1);
qdot(3) = vy;
qdot(4) = z(2);

%transpose qdot
qdot = qdot';

end
```

```
%{
Jose Nino
Q19 RHS file for simple pendulum minimal coordinate approach
%}

function qdot = RHS4(t,q,p)
%rename states to better understand equations
theta = q(1);
dtheta = q(2);

%build qdot matrix to send to ode45 for integration
%equations derived from minimal coordinate apprach using theta
qdot(1) = dtheta;
qdot(2) = -p.g/p.l*sin(theta);
%transpose qdot
qdot = qdot';
end
```





Notice: With a high tolerance the DAE doesn't seem to violate the constraint. However, with a low constraint the error drift is noticiable.



Note: Low tolerance error builds up quickly, while high tolerance error is negligable.